

LAB 4. VIEW COMPONENT & PHÂN TRANG

Hiện tại, trang web của chúng ta đã có giao diện đẹp, nhưng sidebar vẫn còn tĩnh và nếu có hàng trăm sản phẩm, trang chủ sẽ tải rất chậm. Hôm nay, chúng ta sẽ giải quyết hai vấn đề này bằng cách tạo một menu danh mục động với **View Component** và triển khai **Phân trang** hiệu quả.

MỤC TIÊU BÀI HỌC

Kết thúc buổi học, bạn sẽ có khả năng:

1. **Xây dựng một View Component** hoàn chỉnh để tạo ra các phần UI phức tạp, có logic riêng và tái sử dụng được.
2. **Sử dụng LINQ** (Skip, Take, OrderBy) để lấy ra chính xác "lát" dữ liệu cần thiết cho mỗi trang.
3. **Truyền dữ liệu phân trang** từ Controller sang View một cách hiệu quả.
4. **Tạo một Tag Helper tùy chỉnh (Custom Tag Helper)**, một kỹ thuật nâng cao để đóng gói logic tạo HTML phức tạp.
5. **Tích hợp chức năng lọc** sản phẩm theo danh mục vào logic phân trang hiện có.

YÊU CẦU KIẾN THỨC

- Đã hoàn thành bài thực hành số 3.
- Hiểu biết về LINQ (Language Integrated Query).
- Kiến thức cơ bản về vòng đời của một request MVC.

CÔNG CỤ CẦN THIẾT

- Solution SportsStore đã hoàn thành ở buổi 3.

NỘI DUNG

Phần 1: Phân tích lý thuyết

1. View Component là gì?

Hãy coi **View Component** như một "mini-controller" chỉ dành cho việc render một phần của giao diện. Nó mạnh hơn Partial View vì nó có thể chứa logic nghiệp vụ riêng để lấy dữ liệu.

- **Tại sao cần nó?** Sidebar danh mục của chúng ta là một ví dụ hoàn hảo. Nó cần:
 - Lấy danh sách tất cả các danh mục từ database.
 - Hiển thị chúng dưới dạng một danh sách các liên kết.
 - Logic này không thuộc về HomeController (vốn chỉ nên quan tâm đến việc hiển thị sản phẩm) và cần được tái sử dụng ở nhiều nơi.
- **Cấu tạo:** Một View Component bao gồm 2 phần:
 1. **Lớp Logic (Class):** Kế thừa từ ViewComponent, có một phương thức InvokeAsync để xử lý logic và trả về một IViewComponentResult.
 2. **View Giao diện (Razor file):** Một file .cshtml để định nghĩa HTML sẽ được render.

2. Phân trang (Pagination)

Khi có 1000 sản phẩm, chúng ta không thể tải tất cả cùng lúc. Phân trang là kỹ thuật chia 1000 sản phẩm đó thành các "trang" nhỏ hơn (ví dụ: 10 sản phẩm mỗi trang).

- Logic hoạt động:
 - Người dùng yêu cầu trang số 3.
 - Ứng dụng tính toán: "Để đến trang 3, tôi cần bỏ qua (Skip) 2 trang đầu tiên. Mỗi trang có 10 sản phẩm, vậy tôi sẽ Skip($2 * 10 = 20$) sản phẩm."
 - Sau đó, ứng dụng lấy (Take) 10 sản phẩm tiếp theo: Take(10).
 - **LINQ** cung cấp sẵn các phương thức Skip() và Take() cực kỳ hiệu quả cho việc này.

Phần 2: Hướng dẫn thực hành chi tiết

Bước 1: Tạo NavigationMenuViewComponent

Chúng ta sẽ tạo menu danh mục động để đặt vào sidebar.

1. Tạo thư mục cho View Component:
 - Trong project SportsStore.WebUI, tạo một thư mục mới tên là Components.
2. Tạo Lớp Logic NavigationMenuViewComponent.cs:
 - Trong thư mục Components, tạo một class mới tên là NavigationMenuViewComponent.cs.

- Viết code như sau:

```
// SportsStore.WebUI/Components/NavigationViewComponent.cs

using Microsoft.AspNetCore.Mvc;
using SportsStore.Domain;

namespace SportsStore.WebUI.Components;

public class NavigationMenuViewComponent : ViewComponent
{
    private IProductRepository _repository;

    public NavigationMenuViewComponent(IProductRepository repo)
    {
        _repository = repo;
    }

    // Phương thức chính của View Component
    public IViewComponentResult Invoke()
    {
        // Dùng LINQ để lấy ra các danh mục duy nhất và sắp xếp chúng
        var categories = _repository.Products
            .Select(p => p.Category)
            .Distinct()
            .OrderBy(c => c);

        // Trả về View tương ứng, kèm theo danh sách categories
        return View(categories);
    }
}
```

- Giải thích:

- Class phải có tên kết thúc bằng ViewComponent.
- Nó được inject IProductRepository qua DI, giống như Controller.
- Phương thức Invoke lấy tất cả sản phẩm, chỉ chọn ra thuộc tính Category, loại bỏ các giá trị trùng lặp (Distinct), sắp xếp theo thứ tự alphabet, và cuối cùng truyền danh sách này cho View.

3. Tạo View Giao diện Default.cshtml:

- ASP.NET Core có quy ước tìm kiếm View cho Component. Chúng ta sẽ tạo cấu trúc thư mục sau: Views/Shared/Components/NavigationMenu/.
- Trong thư mục Views/Shared, tạo thư mục Components.
- Trong Components, tạo thư mục NavigationMenu (tên của View Component bỏ đi hậu tố ViewComponent).
- Trong NavigationMenu, tạo một file Razor View tên là Default.cshtml.
- Nội dung file:

```
@* Views/Shared/Components/NavigationMenu/Default.cshtml *@

@model IEnumerable<string>

<div class="d-grid gap-2">
    <a asp-controller="Home" asp-action="Index" class="btn btn-outline-secondary">Home</a>

    @foreach (string category in Model)
    {
        <a asp-controller="Home"
            asp-action="Index"
            asp-route-category="@category"
            class="btn btn-outline-secondary">
            @category
        </a>
    }
</div>
```

– Giải thích:

- `asp-route-category="@category"`: Tag Helper này sẽ thêm tham số category vào URL, ví dụ: /Home/Index?category=Soccer.

4. Gọi View Component trong _Layout.cshtml:

- Mở file _Layout.cshtml trong Views/Shared.
- Tìm đến thẻ <aside> (sidebar).
- Thay thế nội dung tĩnh bên trong bằng thẻ gọi View Component:

```
<aside class="col-md-3">
    @await Component.InvokeAsync("NavigationMenu")
</aside>
```

- Bây giờ, mỗi khi layout được render, nó sẽ tự động gọi View Component của chúng ta để tạo ra menu danh mục động.

Bước 2: Triển khai Phân trang trong HomeController

1. Cập nhật HomeController:

- Mở HomeController.cs và sửa lại action Index.

```
// SportsStore.WebUI/Controllers/HomeController.cs

// ... using statements

public class HomeController : Controller
{
    private IProductRepository _repository;
    public int PageSize = 4; // Quy định số sản phẩm trên mỗi trang

    public HomeController(IProductRepository repo)
    {
        _repository = repo;
    }

    // Action Index giờ sẽ nhận tham số category và page
    public IActionResult Index(string? category, int productPage = 1)
    {
        var products = _repository.Products
            // Lọc theo category nếu có
            .Where(p => category == null || p.Category == category)
            .OrderBy(p => p.ProductID)
            .Skip((productPage - 1) * PageSize) // Bỏ qua các sản phẩm ở
trang trước
            .Take(PageSize); // Lấy số sản phẩm tương ứng với trang hiện
tại

        return View(products);
    }
}
```

– Giải thích:

- PageSize = 4: Chúng ta đặt số sản phẩm mỗi trang là 4 để dễ kiểm tra.
- productPage = 1: Tham số này nhận số trang từ URL (ví dụ: /Home/Index?productPage=2). Nếu không có, mặc định là trang 1.
- .Where(p => category == null || p.Category == category): Đây là logic lọc. Nếu category là null (khi xem trang chủ), điều kiện category == null là true, và tất cả sản phẩm sẽ được lấy. Nếu có category (ví dụ: "Soccer"), nó sẽ chỉ lấy các sản phẩm có Category khớp.
- .Skip(...) và .Take(...) thực hiện logic phân trang như đã giải thích ở trên.

2. Chạy thử:

- Nhấn **F5**. Bạn sẽ thấy trang chủ bây giờ chỉ hiển thị 4 sản phẩm đầu tiên.
- Bấm vào một danh mục ở sidebar (ví dụ "Soccer"). URL sẽ đổi thành /?category=Soccer và bạn sẽ chỉ thấy các sản phẩm thuộc danh mục đó.
- Thử thay đổi URL thủ công thành /?productPage=2. Bạn sẽ thấy 4 sản phẩm tiếp theo. Phân trang và lọc đã hoạt động! Nhưng người dùng không thể tự gõ URL, chúng ta cần tạo các nút chuyển trang.

Bước 3: Tạo Tag Helper tùy chỉnh cho Phân trang

Chúng ta sẽ tạo một tag helper <div page-model="..." ...></div> để nó tự động sinh ra các liên kết <a>1, <a>2, <a>3...

1. Tạo ViewModel cho Phân trang:

- Tạo thư mục Models trong SportsStore.WebUI.
- Trong thư mục Models, tạo class PagingInfo.cs:

```
// SportsStore.WebUI/Models/PagingInfo.cs
namespace SportsStore.WebUI.Models;

public class PagingInfo
{
    public int TotalItems { get; set; }
    public int ItemsPerPage { get; set; }
    public int CurrentPage { get; set; }
    public int TotalPages => (int)Math.Ceiling((decimal)TotalItems /
```

```
ItemsPerPage);  
}
```

- Class này chứa tất cả thông tin cần thiết để render các nút phân trang.

2. Tạo một ViewModel tổng hợp:

- Chúng ta cần truyền cả danh sách sản phẩm và thông tin phân trang tới View. Cách tốt nhất là tạo một ViewModel tổng hợp.
- Trong SportsStore.WebUI/Models, tạo class ProductsListViewModel.cs:

```
using SportsStore.Domain;  
  
namespace SportsStore.WebUI.Models;  
  
public class ProductsListViewModel  
{  
    public IEnumerable<Product> Products { get; set; } =  
        Enumerable.Empty<Product>();  
    public PagingInfo PagingInfo { get; set; } = new();  
}
```

3. Cập nhật HomeController để sử dụng ViewModel:

- Quay lại HomeController.cs và sửa action Index:

```
// Thêm using cho ViewModel  
using SportsStore.WebUI.Models;  
  
// ... bên trong class HomeController  
public IActionResult Index(string? category, int productPage = 1)  
{  
    return View(new ProductsListViewModel  
    {  
        Products = _repository.Products  
            .Where(p => category == null || p.Category == category)  
            .OrderBy(p => p.ProductID)  
            .Skip((productPage - 1) * PageSize)  
            .Take(PageSize),  
        PagingInfo = new PagingInfo  
        {  
            CurrentPage = productPage,  
            ItemsPerPage = PageSize,  

```

```
        TotalItems = category == null
            ? _repository.Products.Count()
            : _repository.Products.Where(e => e.Category ==
category).Count()
    }
    });
}
```

– Giải thích:

- Giờ đây action Index trả về một ProductsListViewModel chứa cả danh sách sản phẩm cho trang hiện tại và thông tin phân trang (tổng số sản phẩm, trang hiện tại...).
- TotalItems được tính toán dựa trên việc có lọc theo danh mục hay không.

4. Tạo lớp Tag Helper:

- Tạo thư mục Infrastructure trong SportsStore.WebUI.
- Trong đó, tạo class PageLinkTagHelper.cs:

```
// SportsStore.WebUI/Infrastructure/PageLinkTagHelper.cs

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Rendering;
using Microsoft.AspNetCore.Mvc.Routing;
using Microsoft.AspNetCore.Mvc.ViewFeatures;
using Microsoft.AspNetCore.Razor.TagHelpers;
using SportsStore.WebUI.Models;

namespace SportsStore.WebUI.Infrastructure;

[HtmlTargetElement("div", Attributes = "page-model")]
public class PageLinkTagHelper : TagHelper
{
    private IUrlHelperFactory urlHelperFactory;

    public PageLinkTagHelper(IUrlHelperFactory helperFactory)
    {
        urlHelperFactory = helperFactory;
    }
}
```



```
[ViewContext]
[HtmlAttributeNotBound]
public ViewContext? ViewContext { get; set; }

public PagingInfo? PageModel { get; set; }
public string? PageAction { get; set; }

public override void Process(TagHelperContext context, TagHelperOutput
output)
{
    if (ViewContext != null && PageModel != null)
    {
        IUrlHelper urlHelper =
urlHelperFactory.GetUrlHelper(ViewContext);
        TagBuilder result = new TagBuilder("div");
        for (int i = 1; i <= PageModel.TotalPages; i++)
        {
            TagBuilder tag = new TagBuilder("a");
            tag.Attributes["href"] = urlHelper.Action(PageAction, new {
productPage = i });
            tag.InnerHtml.Append(i.ToString());
            if (i == PageModel.CurrentPage)
            {
                tag.AddCssClass("btn btn-primary");
            }
            else
            {
                tag.AddCssClass("btn btn-outline-primary");
            }
            result.InnerHtml.AppendHtml(tag);
        }
        output.Content.AppendHtml(result.InnerHtml);
    }
}
```

- Đây là phần phức tạp nhất. Tóm tắt:
 - [HtmlTargetElement("div", Attributes = "page-model")]: Báo cho Razor biết Tag Helper này sẽ được kích hoạt trên thẻ <div> nào có thuộc tính page-model.

- PageModel và PageAction: Là các thuộc tính mà chúng ta sẽ truyền vào từ View.
- Process method: Đây là nơi logic xảy ra. Nó lặp từ 1 đến tổng số trang, tạo ra một thẻ <a> cho mỗi trang, đặt href chính xác, thêm class CSS để làm nổi bật trang hiện tại, và cuối cùng nhét tất cả vào thẻ div mẹ.

5. Kích hoạt Tag Helper:

- Mở file _ViewImports.cshtml trong thư mục Views.
- Thêm dòng sau để ứng dụng nhận biết các Tag Helper tùy chỉnh trong project:

```
@addTagHelper *, SportsStore.WebUI
```

6. Sử dụng Tag Helper trong View:

- Mở file Index.cshtml trong Views/Home.
- Thay đổi khai báo model ở đầu file:

```
@model ProductsListViewModel
```

- Thay đổi vòng lặp foreach để duyệt Model.Products:

```
@foreach (var p in Model.Products)
{
}
```

- Thêm đoạn code sau vào cuối file để hiển thị các nút phân trang:

```
<div page-model="@Model.PagingInfo" page-action="Index" class="btn-group float-end">
</div>
```

7. Chạy và kiểm tra:

- Nhấn F5. Bây giờ bạn sẽ thấy các nút phân trang ở góc dưới bên phải.
- Bấm vào các nút đó, bạn sẽ thấy danh sách sản phẩm và URL thay đổi tương ứng.
- Thử bấm vào một danh mục, rồi bấm vào các nút chuyển trang. Bạn sẽ thấy nó hoạt động đúng: chỉ phân trang cho các sản phẩm trong danh mục đó.

Phần 3: Tổng kết & Mở rộng

Kết quả

Bạn đã thực hiện được một bước tiến lớn trong việc xây dựng ứng dụng web chuyên nghiệp:

- **Sidebar động:** Sidebar giờ đây tự động hiển thị các danh mục có trong dữ liệu, giúp người dùng dễ dàng điều hướng.
- **Hiệu suất được cải thiện:** Trang web chỉ tải 4 sản phẩm mỗi lần, giúp tăng tốc độ tải trang đáng kể khi dữ liệu lớn lên.
- **Trải nghiệm người dùng tốt hơn:** Giao diện phân trang rõ ràng giúp người dùng dễ dàng khám phá tất cả các sản phẩm.
- **Mã nguồn có khả năng tái sử dụng cao:** Cả View Component và Tag Helper đều là những thành phần có thể dễ dàng tái sử dụng ở các trang khác.

Bài tập mở rộng

1. **Làm nổi bật danh mục đang chọn:** Trong View Component `NavigationMenuViewComponent`, hãy tìm cách truyền danh mục hiện tại đang được chọn (lấy từ URL) vào View. Sau đó, trong file `Default.cshtml`, sử dụng một câu lệnh `if` để thêm một class CSS (ví dụ `btn-primary`) cho nút danh mục đang được chọn.
2. **Tùy chỉnh Tag Helper:** Thêm các thuộc tính mới vào `PageLinkTagHelper` để có thể tùy chỉnh các class CSS từ View. Ví dụ: thêm thuộc tính `public string PageClass { get; set; }`, `public string PageClassSelected { get; set; }` và sử dụng chúng thay vì hard-code `btn-primary` và `btn-outline-primary`.

TÀI LIỆU THAM KHẢO

- [1]. View Components in ASP.NET Core: Microsoft Docs - View Components
- [2]. Authoring Tag Helpers in ASP.NET Core: Microsoft Docs - Authoring Tag Helpers
- [3]. LINQ Query Examples (Skip, Take): LINQ Samples - Skip & Take